

CassanovaL — Panduan Menambahkan Agent Baru (Tanpa AI)

Dokumen ini menjelaskan langkah-langkah lengkap untuk menambahkan **chat agent baru** ke sistem CassanovaL secara manual. Ikuti urutan ini secara berurutan.

Checklist Singkat

- [] 1. Buat file tools → `tools/<key>_tools.py`
- [] 2. Buat file agent → `agents/<key>_agent.py`
- [] 3. Daftarkan di `router.py` (3 tempat)
- [] 4. Daftarkan di `static/index/data.jsx` (3 tempat)
- [] 5. Tambahkan avatar ke `static/avatars/`

Konsep Dasar

Komponen	Fungsi
<code>tools/<key>_tools.py</code>	Fungsi-fungsi yang bisa dipanggil agent (simpan data, baca file, dsb)
<code>agents/<key>_agent.py</code>	Kepribadian, instruksi, dan kumpulan tools agent
<code>router.py</code>	Mengarahkan pesan user ke agent yang tepat
<code>static/index/data.jsx</code>	Konfigurasi UI: nama, warna, ikon, chip prompt
<code>static/avatars/</code>	Foto avatar agent

Istilah kunci: - `key` = nama pendek agent dalam huruf kecil, contoh: `fitness`, `journal`, `stock` -
`AgentName` = nama karakter agent, contoh: `Lavoisier`, `Dostoyevsky`

Langkah 1 — Buat File Tools

Buat file `tools/<key>_tools.py`. Tools adalah fungsi Python biasa yang didekorasi `@tool`.

```
# tools/<key>_tools.py

import json
import os
from datetime import datetime
from langchain.tools import tool

DATA_FILE = "data/<key>.json" # ganti <key> sesuai agent

def _load() → dict:
    if not os.path.exists(DATA_FILE):
        return {}
    with open(DATA_FILE, "r", encoding="utf-8") as f:
        return json.load(f)

def _save(data: dict):
    os.makedirs(os.path.dirname(DATA_FILE), exist_ok=True)
    with open(DATA_FILE, "w", encoding="utf-8") as f:
        json.dump(data, f, ensure_ascii=False, indent=2)

@tool
def example_tool(input: str) → str:
    """Deskripsi singkat apa yang dilakukan tool ini. Deskripsi ini dibaca oleh LLM."""
    data = _load()
    # ... logika tool
    _save(data)
    return "Hasil dalam string"

# Kumpulkan semua tools dalam satu list - dipakai di agent file
MY_TOOLS = [example_tool]
```

Aturan tools: - Nama fungsi = nama tool yang dikenal LLM. Buat deskriptif (`add_expense` , `list_tasks`).
 - Docstring wajib ada — LLM membacanya untuk tahu kapan memakai tool. - Return value harus `str` (bukan dict, bukan list mentah — stringify dulu kalau perlu). - Simpan data di `data/<key>.json` supaya konsisten dengan agent lain.

Langkah 2 — Buat File Agent

Buat file `agents/<key>_agent.py` . File ini mendefinisikan kepribadian dan tools agent.

```
# agents/<key>_agent.py

from agents.base import build_agent
from tools.<key>_tools import MY_TOOLS # import tools yang sudah dibuat

SYSTEM_PROMPT = """Kamu adalah [AgentName] - [deskripsi kepribadian 1-2 kalimat].

## APA YANG BISA KAMU LAKUKAN

1. **[KAPABILITAS 1]** - Penjelasan singkat.
2. **[KAPABILITAS 2]** - Penjelasan singkat.

## FORMAT RESPONS

[Jelaskan format output yang diharapkan. Gunakan template markdown jika perlu.]

## PERILAKU

- [Aturan 1]
- [Aturan 2]

Gunakan Bahasa Indonesia secara default."""

def create_<key>_agent():
    return build_agent(
        SYSTEM_PROMPT,
        MY_TOOLS,
        model="mistral-large-latest", # atau "mistral-small-latest" untuk agen ringan
        max_tokens=2048,
    )
```

Pilihan model:

Model	Kapan dipakai
<code>mistral-large-latest</code>	Agent utama yang butuh reasoning kompleks (default)
<code>mistral-small-latest</code>	Agent sederhana, CRUD only, atau yang sering dipanggil

Parameter `build_agent` :

```
build_agent(
    system_prompt: str, # instruksi kepribadian
    tools: list, # list tool functions
    temperature: float, # 0.0 = deterministik, 0.7 = kreatif (default 0.2)
    max_tokens: int, # panjang output maksimal (default 2048)
    model: str, # nama model Mistral
)
```

Langkah 3 — Daftarkan di `router.py`

Buka `router.py` dan edit **3 tempat** berikut:

3a. `_RECURSION_LIMITS` (baris ~45)

Tambahkan entry untuk agent baru. Nilai ini membatasi berapa kali agent boleh memanggil tools sebelum berhenti.

```
_RECURSION_LIMITS = {
    "task": 8,
    "notes": 8,
    # ... agent lain ...
    "<key>": 8, # ← tambahkan di sini
}
```

Panduan nilai: - **6** — agent yang jarang pakai tools (news, journal) - **8** — agent standar CRUD (task, notes, budget) - **10** — agent yang butuh banyak tool calls berantai (coding, research, fitness)

3b. `AGENT_REGISTRY` (baris ~59)

Tambahkan deskripsi agent untuk classifier. Deskripsi ini menentukan kapan router mengarahkan pesan ke agent ini.

```
AGENT_REGISTRY = {
    "task": "Managing to-do lists, tasks, reminders, and deadlines",
    # ... agent lain ...
    "<key>": "Deskripsi singkat kapan agent ini dipanggil – tulis dalam bahasa Inggris atau Ind",
}
```

Tulis deskripsi yang spesifik agar tidak bertabrakan dengan agent lain. Sertakan kata-kata kunci yang biasa dipakai user.

3c. `_load_agent()` (baris ~138)

Tambahkan blok `elif` untuk lazy-load agent baru:

```
def _load_agent(self, name: str):
    if name not in self._agents:
        if name == "task":
            from agents.task_agent import create_task_agent
            self._agents[name] = create_task_agent()
        # ... agent lain ...
        elif name == "<key>": # ← tambahkan di sini
            from agents.<key>_agent import create_<key>_agent
            self._agents[name] = create_<key>_agent()
    return self._agents[name]
```

3d. `CLASSIFY_PROMPT` — Contoh routing (baris ~71, opsional tapi disarankan)

Tambahkan 3–5 contoh pesan dan routing yang benar di blok `Examples of correct routing`:

- "pesan yang relevan dengan agent ini" → `<key>`
- "contoh lain" → `<key>`

Contoh yang baik membantu classifier memilih agent yang tepat.

Langkah 4 — Daftarkan di `static/index/data.jsx`

Buka `static/index/data.jsx` dan edit **3 tempat**:

4a. Objek `AGENTS` (baris ~4)

Tambahkan entry agent baru. Ikuti urutan `issue` yang benar (Roman numeral).

```
const AGENTS = {
  task: { ... },
  // ... agent lain ...
  <key>: {
    name: 'AgentName',           // nama karakter, contoh: 'Lavoisier'
    sub: 'Label Singkat',        // subtitle, contoh: 'Fitness & Health'
    hue: 'var(--hue-<key>)',      // CSS variable warna (lihat langkah 4d)
    issue: 'X.',                 // nomor Roman sesuai urutan
    cluster: 'personal',         // cluster: 'personal' | 'research' | 'academic' | 'trading'
    tagline: 'Satu kalimat deskripsi gaya sastra tentang agent ini.',
    greeting: 'Pesan sambutan yang muncul saat user pertama kali membuka agent ini.',
    Ico: () => { const {IcoXxx} = window.Icons; return <IcoXxx/>; }, // ikon SVG
    // url: '/halaman-baru',      // OPSIONAL: jika agent punya halaman tersendiri (external 1
  },
};
```

Field `cluster` — menentukan tab filter di sidebar:

Nilai	Deskripsi
<code>personal</code>	Kehidupan sehari-hari (Alfred, Mansa, Lavoisier, dll)
<code>research</code>	Riset dan informasi (Najwa)
<code>academic</code>	Belajar dan coding (Cicero, Linus)
<code>trading</code>	Pasar keuangan — khusus pipeline Crew Mode

Field `url` (opsional) — jika diisi, klik pada agent di sidebar akan membuka URL tersebut di tab baru, bukan membuka chat tab. Gunakan untuk agent yang punya halaman tersendiri (contoh: Mansa → `/finance`). Roster row akan menampilkan  sebagai pengganti status dot.

Icon yang tersedia (lihat `static/index/icons.jsx`): `IcoCheck`, `IcoFeather`, `IcoNewspaper`, `IcoCode`, `IcoCalendar`, `IcoCoin`, `IcoHeart`, `IcoBook`, `IcoLamp`, `IcoChart`, `IcoSearch`, `IcoUser`

4b. Array `AGENT_ORDER` (baris ~69)

Tambahkan key agent ke array sesuai urutan tampil di sidebar:

```
const AGENT_ORDER = ['task', 'notes', 'news', 'coding', 'schedule', 'budget', 'fitness', 'journal', 'davinci'];
```

4c. Objek `CHIPS` (baris ~71)

Tambahkan 4 tombol shortcut prompt yang muncul di bawah input chat:

```
const CHIPS = {
  // ... agent lain ...
  <key>: ['Prompt cepat 1', 'Prompt cepat 2', 'Prompt cepat 3', 'Prompt cepat 4'],
};
```

Pilih prompt yang paling sering dipakai user untuk agent ini.

4d. CSS variable warna — `static/index/styles.css`

Buka `static/index/styles.css` dan tambahkan CSS variable warna di blok `:root`:

```
:root {
  /* ... variable lain ... */
  --hue-<key>: #RRGGBB; /* ganti dengan warna hex yang sesuai */
}
```

Konvensi warna yang sudah dipakai (hindari duplikasi): - Alfred (task): biru tua - Cicero (notes): hijau tua - Najwa (news): merah - Linus (coding): ungu - Miyamoto (schedule): biru langit - Mansa (budget): emas - Lavoisier (fitness): oranye - Dostoyevsky (journal): coklat - Da Vinci (davinci): kuning

Langkah 5 — Tambahkan Avatar

Simpan gambar avatar ke `static/avatars/<key>.jpg` (atau `.png`).

- Ukuran ideal: **400×400 px** atau persegi
- Format: JPG atau PNG
- Nama file harus sama persis dengan `key` agent

Jika tidak ada avatar, agent tetap berjalan — hanya tampilan UI saja yang tidak ada foto.

Verifikasi

Setelah semua langkah selesai, jalankan server dan tes:

```
$env:PYTHONUTF8=1; python server.py
```

Buka `http://localhost:8000` di browser, lalu:

1. **Cek sidebar** — Agent baru muncul di daftar dengan nama dan ikon yang benar.
2. **Kirim pesan relevan** — Router harus memilih agent baru secara otomatis (bukan agent lain).
3. **Test tools** — Minta agent melakukan aksi yang memanggil tools (simpan data, baca data, dll).
4. **Cek direct routing** — Klik agent langsung dari sidebar, kirim pesan.

Menambahkan CrewAI Multiagent Pipeline

Jika ingin menambahkan **pipeline background** (bukan chat agent), edit `crewai_agents.py`:

Struktur Dasar Pipeline

```
from crewai import Agent, Task, Crew, Process
from langchain_mistralai import ChatMistralAI

def build_llm(model="mistral-large-latest"):
    return ChatMistralAI(model=model, temperature=0.3, api_key=os.getenv("MISTRAL_API_KEY"))

class MyNewPipeline:
    def kickoff(self, topic: str) → dict:
        llm_small = build_llm("mistral-small-latest")
        llm_large = build_llm("mistral-large-latest")

        # Definisikan agents
        agent1 = Agent(
            role="Peran Agent 1",
            goal="Tujuan spesifik agent ini",
            backstory="Latar belakang kepribadian agent",
            llm=llm_small,
            verbose=True,
        )

        # Definisikan tasks
        task1 = Task(
            description=f"Instruksi detail untuk task ini. Topik: {topic}",
            expected_output="Apa yang harus dihasilkan (format, panjang, dsb)",
            agent=agent1,
            output_file="AI Data/MyAgent/output1.txt", # opsional
        )

        # Jalankan crew
        crew = Crew(
            agents=[agent1],
            tasks=[task1],
            process=Process.sequential, # atau Process.hierarchical
            verbose=True,
        )
        result = crew.kickoff()
        return {"output": str(result)}
```

Daftarkan Pipeline di `server.py`

Buka `server.py` dan tambahkan kondisi baru di blok `crew_type` dalam fungsi background job:

```
# Di dalam fungsi _run_job() atau background thread
elif crew_type == "mynewpipeline":
    from crewai_agents import MyNewPipeline
    pipeline = MyNewPipeline()
    result = pipeline.kickoff(topic)
    outputs["result.md"] = str(result)
```

Nilai `crew_type` yang sudah ada: - `"research"` — Ibn Al-Haytham 7-agent pipeline - `"dataanalyst"`
— DataAnalyst 3-agent pipeline - `"scraper"` — Social Scraper (ScrapeGraphAI + SummarizerAgent)

Social Scraper Pipeline (`crew_type: "scraper"`) berbeda dari pipeline CrewAI biasa — ia tidak menggunakan `crewai_agents.py` , melainkan: 1. `social_scraper/agents/scrapegraph_harvester.py` — ScrapeGraphAI v2 + Mistral untuk harvest konten trending 2. `social_scraper/agents/summarizer_agent.py` — Mistral untuk generate ringkasan per platform

Parameter scraper via `POST /api/crew/kickoff` :

```
{
  "crew_type": "scraper",
  "topic": "kata kunci opsional",
  "platforms": ["youtube", "tiktok", "reddit"],
  "translate": false
}
```

Jika `platforms` null, default ke `["youtube", "tiktok", "facebook", "instagram"]` .

Troubleshooting Umum

Masalah	Kemungkinan Penyebab	Solusi
Agent tidak dipilih router	Deskripsi di <code>AGENT_REGISTRY</code> tidak spesifik	Tambahkan kata kunci yang lebih khas
<code>KeyError</code> saat chat	Key belum ada di <code>_RECURSION_LIMITS</code> atau <code>_chat_histories</code>	Pastikan key ada di ketiga dict di <code>router.py</code>
Tool tidak dipanggil agent	Docstring tool kurang deskriptif	Perjelas docstring dengan contoh kapan tool ini dipakai
Agent tidak muncul di sidebar	Key belum ada di <code>AGENT_ORDER</code> di <code>data.jsx</code>	Tambahkan key ke array <code>AGENT_ORDER</code>
Error <code>MISTRAL_API_KEY</code>	File <code>.env</code> tidak ada atau key salah	Pastikan <code>.env</code> ada di root project dengan key yang valid
Tool return error JSON	Tool mengembalikan dict, bukan string	Tambahkan <code>return json.dumps(result)</code> di akhir tool

Pola Data & History (Update 2026-06-17)

Tiga pola berikut baru ditambahkan/diperbaiki dan berguna sebagai contoh saat membuat agent baru yang punya "history".

1. Mansa — urutkan transaksi terbaru dengan `created_at`

Setiap transaksi disimpan dengan `created_at = "YYYY-MM-DD HH:MM"` (lihat `add_income / add_expense` di `tools/budget_tools.py`). Endpoint `GET /api/finance/dashboard` dan `GET /api/budget/summary` mengurutkan `recent_transactions` berdasarkan `created_at`, **bukan** `date` saja:

```
key=lambda x: (x.get("created_at") or (x.get("date", "") + " 00:00"))
```

Pelajaran: kalau mengurutkan "yang terbaru", pakai timestamp lengkap (tanggal + **jam**). Mengurutkan dengan tanggal saja membuat entri baru di hari yang sama tenggelam karena `sorted()` Python bersifat *stable*. Dashboard `/finance` juga auto-refresh tiap 5 detik (`setInterval(loadDashboard, 5000)`).

2. Lavoisier — backfill history dari file markdown ke JSON

`tools/food_tools.py` menyimpan log ke `data/food_log.json` dan me-mirror tiap hari ke `FoodSummary_YYYY-MM-DD.md` di folder Lavoisier Agent. Untuk memunculkan history lama, `server.py` punya `_sync_food_logs_from_md()` yang membaca balik file markdown ke `food_log.json` untuk tanggal yang belum ada. `_lavoisier_dirs()` memindai **dua** folder: vault aktif (`AI Data/My AI/Lavoisier Agent`) dan folder lama (`AI Data/Lavoisier Agent`). Sync dipanggil non-blocking saat startup dan via `POST /api/sync/food-logs`.

Pelajaran: untuk agent dengan mirror markdown ↔ JSON, sediakan sync dua arah dan pindai semua lokasi history (lama + baru). Jangan timpa hari yang sudah ada.

3. Dostoyevsky — kembalikan history percakapan

Reflecta menulis jurnal secara percakapan (`**Kamu:** / **Journal:**`) ke `Journal_YYYY-MM-DD.md` di `<OBSIDIAN_VAULT_PATH>/Dostoyevsky Agent`. Dua tempat memunculkan history: - **Chat agent** (`agents/dostoyevsky_agent.py`): **Phase 0** memanggil `list_journal_entries` + `get_mood_history` sekali di awal sesi, lalu `read_journal_entry / search_journal` saat perlu. - **Dashboard** (`GET /api/journal/dashboard`): mempertahankan isi percakapan sebagai body entri; label pembicara dan header `## HH:MM · JOURNAL` hanya dibersihkan untuk preview/word-count.

Pelajaran: kalau agent journaling-mu menyimpan dialog, jangan buang baris user saat menampilkan history — itu justru isi jurnalnya. Suruh agent memuat history di awal sesi agar terasa berkesinambungan.

Referensi File

File	Fungsi
agents/base.py	<code>build_agent()</code> — builder utama semua chat agent
router.py	Supervisor router — klasifikasi + dispatch ke agent
server.py	FastAPI server — HTTP endpoints + background jobs
crewai_agents.py	Pipeline CrewAI Ibn Al-Haytham + DataAnalyst
social_scraper/agents/scrapegraph_harvester.py	Social Scraper — ScrapeGraphAI v2 harvester
social_scraper/agents/summarizer_agent.py	Social Scraper — Mistral per-platform summarizer
static/index/data.jsx	Konfigurasi UI agent: AGENTS, AGENT_CLUSTERS, CHIPS
static/index/icons.jsx	Semua komponen ikon SVG yang tersedia
static/index/styles.css	CSS variables termasuk warna per agent + cluster
static/finance/index.html	Mansa Finance Dashboard — halaman tersendiri di /finance
CLAUDE.md	Dokumentasi arsitektur lengkap
DOCUMENTATION.md	Referensi kode detail